Author: Pankaj Yadav
Date: 04/05/2026
Description:
Data Prep and Business intelligence

Week 3 Assignment

In this assignment, you will be using data on the Los Angeles Dodgers Major League Baseball (MLB) team located here: dodgers.csv. Use this data to make a recommendation to management on how to improve attendance. Tell a story with your analysis and clearly explain the steps you take to arrive at your conclusion. This is an open-ended question, and there is no one right answer. You are welcome to do additional research and/or use domain knowledge to assist your analysis, but clearly state any assumptions you make.

In [1]:
```
# Import Libraries
import pandas as pd
from pandas.tseries.holiday import USFederalHolidayCalendar
import matplotlib.pyplot as plt
```

Import the data

The first step to generate insights is to understand your data. And to understand the data you not just have to understand the variables but also the story behind that data. If you dont know what titanic was or what a baseball game is, its difficult to find patterns. I still believe pandas a good in reading data, hence using read_csv function from pandas to read csv file in to a data frame and using head function to read the first 5 rows from it.

In [2]:
```
# Load the dataset
mlb_data = pd.read_csv('data//dodgers-2022.csv')

# display first few rows to check if data is loaded correctly
mlb_data.head(5)
```

Out[2]:

	month	day	attend	day_of_week	opponent	temp	skies	day_night	cap	shirt	fireworks	bobblehead
0	APR	10	56000	Tuesday	Pirates	67	Clear	Day	NO	NO	NO	NO
1	APR	11	29729	Wednesday	Pirates	58	Cloudy	Night	NO	NO	NO	NO
2	APR	12	28328	Thursday	Pirates	57	Cloudy	Night	NO	NO	NO	NO
3	APR	13	31601	Friday	Padres	54	Cloudy	Night	NO	NO	YES	NO
4	APR	14	46549	Saturday	Padres	57	Cloudy	Night	NO	NO	NO	NO

Based on the sample data printed above, we can easily interpret that the column `month` represents the Month and the `day` is that day of the month. Now that we have the year in datasets name as `2022` we can use it to create a single field as `game_date` and use that field only to represent these three (month, day and day of the week) and drop them from the dataset. I am assuming the variable `attend` represents the total number of people attended the game that day irrespective of their type, means, if they were hard core fan or home team or opponent team or joined as a promo etc. This is our target variable. `Opponent` captures the opponent the dodgers team had their match with, which definitely affects the attendance based on teams popularity, rivalry or player favs. `Temp` variable most probably represents the temperature on that specific day, and after looking at the values, most probably is in fahrenheit. `Skies` captures if the sky was clear or cloudy. This needs some cleaning as `Clear` value have some trailing white spaces. day_night variable is obvious. The `cap` , `shirt` and `bobblehead` just represent the promotional give always present on the game day. `Fireworks` , if there were fireworks on the game night. Also, I am assuming that the games were played within US and that public holidays just capture US federal holidays not school breaks etc.

If you look beyond the variables now, you will see, the day of the week, if its saturday or sunday, it capture leisure-time effect, so weekend vs Weekday will matter for sure. Similarly, between day and night, there is a straight distinction of work obligations in day time vs family time in evenings but promotions could affect the outcomes. Holidays could also affect the numbers to as now we have dates, will add another column to check the dates fall on popular federal holidays.

Below, i am hardcoding year as 2022 and then using to_datetime function from pandas library to convert it in to a date string. Later I am formatting that date in to a numeric date format creating the new variable `game_date` .

In [3]:
```
mlb_data['game_date'] = pd.to_datetime('2022-' + mlb_data['month'].str[:3] +
                                       '-' + mlb_data['day'].astype(str),
                                       format='%Y-%b-%d')
```

```
# Lets check if we can fetch the day of week so that we can consolidate the three columns in one.
mlb_data['day_of_week_new'] = mlb_data['game_date'].dt.day_name()

# lets add a column public_holiday to check if the game was played on a public holiday or not.
# For simplicity, we will consider only the major public holidays in the US.
calendar = USFederalHolidayCalendar()
holidays = calendar.holidays(start=mlb_data['game_date'].min(),
                             end=mlb_data['game_date'].max())

mlb_data['public_holiday'] = mlb_data['game_date'].isin(holidays)

mlb_data.head(5)
```

Out[3]:

	month	day	attend	day_of_week	opponent	temp	skies	day_night	cap	shirt	fireworks	bobblehead	game_date
0	APR	10	56000	Tuesday	Pirates	67	Clear	Day	NO	NO	NO	NO	2022-04-10
1	APR	11	29729	Wednesday	Pirates	58	Cloudy	Night	NO	NO	NO	NO	2022-04-11
2	APR	12	28328	Thursday	Pirates	57	Cloudy	Night	NO	NO	NO	NO	2022-04-12
3	APR	13	31601	Friday	Padres	54	Cloudy	Night	NO	NO	YES	NO	2022-04-13
4	APR	14	46549	Saturday	Padres	57	Cloudy	Night	NO	NO	NO	NO	2022-04-14

From the name of the input data file I assumed the year will be `2022`, but while comparing the `day_the_week` in data frame from the newly derived `day_of_week_new`, I realized, the days were not matching with the dates from `2022`. So I did a backtrack and found out that the dates matches exactly with `2018` & `2012`. I assume the year in name is closely matches with `2012` and typed wrongly as 2022. So Will go ahead with `2012` as year instead of `2018`. Not significant but just wanted to add the right year.

Also, wanted to add a variable that could check if there were any public holidays. They mostly don't impact the outcomes unless they are adjacent to weekends. But adding them to check if there were any impacts.

Instead of treating all the promotional variables separately, I am adding a flag to make sure if there was any promo that day or not.

In [4]:
```
mlb_data['game_date'] = pd.to_datetime('2012-' + mlb_data['month'].str[:3] +
                                       '-' + mlb_data['day'].astype(str),
                                       format='%Y-%b-%d')

#Lets check if we can fetch the day of week so that we can consolidate the three columns in one.
mlb_data['day_of_week_new'] = mlb_data['game_date'].dt.day_name()

#lets add a column public_holiday to check if the game was played on a public
#holiday or not. For simplicity, we will consider only the major public holidays in the US.

calendar = USFederalHolidayCalendar()
holidays = calendar.holidays(start=mlb_data['game_date'].min(),
                             end=mlb_data['game_date'].max())

mlb_data['public_holiday'] = mlb_data['game_date'].isin(holidays)

# Add promotional flag
mlb_data['promo_flag'] = ((mlb_data['cap'] == 'YES')
                          | (mlb_data['shirt'] == 'YES')
                          | (mlb_data['bobblehead'] == 'YES')
                          | (mlb_data['fireworks'] == 'YES'))

# stripping whitespaces from skies column
mlb_data['skies'] = mlb_data['skies'].str.strip()

# Weekend Flag
mlb_data['weekend_flag'] = mlb_data['day_of_week_new'].isin(['Saturday', 'Sunday'])

# print top 5 rows
mlb_data.head(5)
```

Out[4]:

	month	day	attend	day_of_week	opponent	temp	skies	day_night	cap	shirt	fireworks	bobblehead	game_date
0	APR	10	56000	Tuesday	Pirates	67	Clear	Day	NO	NO	NO	NO	2012-04-10
1	APR	11	29729	Wednesday	Pirates	58	Cloudy	Night	NO	NO	NO	NO	2012-04-11
2	APR	12	28328	Thursday	Pirates	57	Cloudy	Night	NO	NO	NO	NO	2012-04-12
3	APR	13	31601	Friday	Padres	54	Cloudy	Night	NO	NO	YES	NO	2012-04-13
4	APR	14	46549	Saturday	Padres	57	Cloudy	Night	NO	NO	NO	NO	2012-04-14

In [5]:
```
# lets drop the original month, day, and day of week columns as we have consolidated them into one column.
mlb_data.drop(columns=['month', 'day','day_of_week_new'], inplace=True)
mlb_data.head(5)
```

Out[5]:

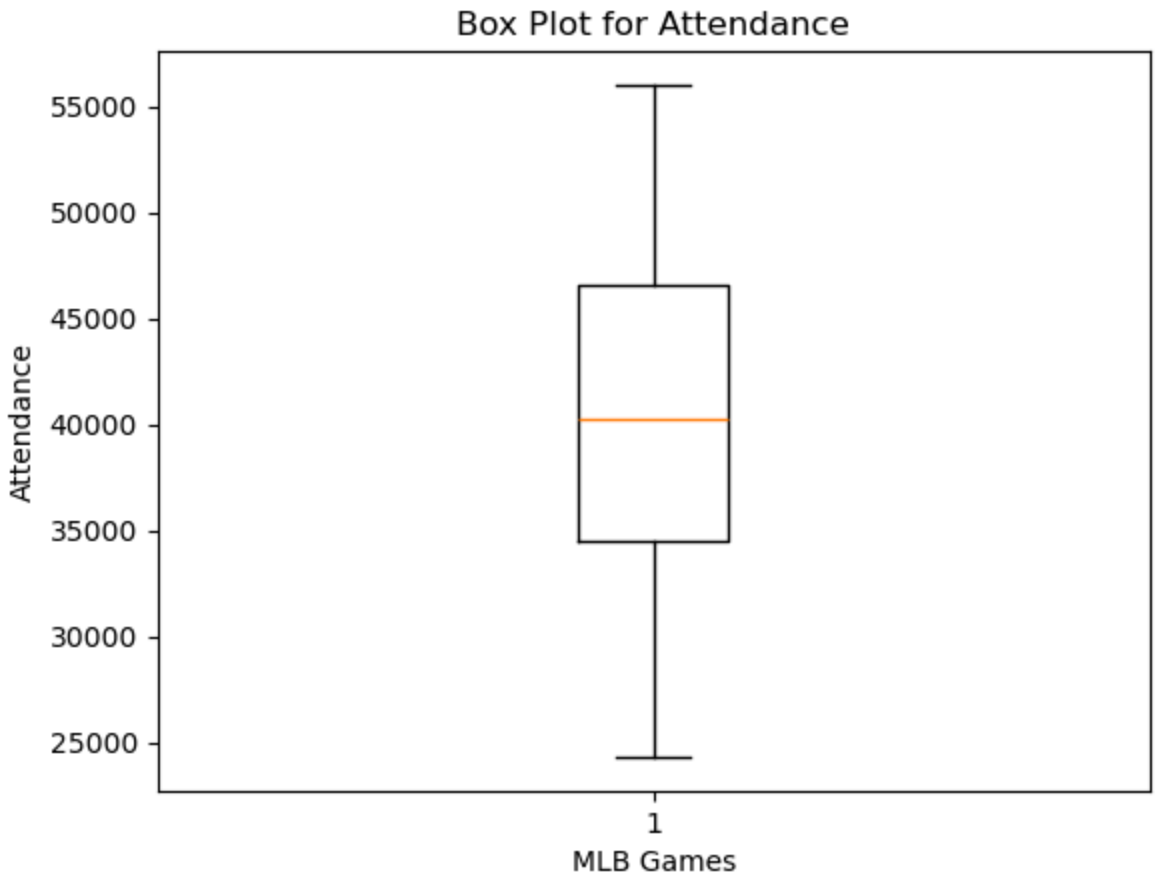
	attend	day_of_week	opponent	temp	skies	day_night	cap	shirt	fireworks	bobblehead	game_date	public_holic
0	56000	Tuesday	Pirates	67	Clear	Day	NO	NO	NO	NO	2012-04-10	Fa
1	29729	Wednesday	Pirates	58	Cloudy	Night	NO	NO	NO	NO	2012-04-11	Fa
2	28328	Thursday	Pirates	57	Cloudy	Night	NO	NO	NO	NO	2012-04-12	Fa
3	31601	Friday	Padres	54	Cloudy	Night	NO	NO	YES	NO	2012-04-13	Fa
4	46549	Saturday	Padres	57	Cloudy	Night	NO	NO	NO	NO	2012-04-14	Fa

I am creating a simple box plot to look at the attendance distribution. The goal is to understand what “normal attendance” looks like and how much it varies across games. I am using matplotlib library and pyplot.boxplot function.

In [6]:
```
# overall attendance distribution

plt.boxplot(mlb_data['attend'])
plt.title('Box Plot for Attendance')
plt.ylabel('Attendance')
```

```
plt.xlabel('MLB Games')
plt.show()
```



The box plot above indicates that attendance is spread across a wide range of values, with a `median` near `40,000` fans. Most games appear to fall between roughly `34,000` and `46,000` attendees, while the full range extends from about `24,000` to `56,000`. This variation suggests that attendance is not uniform and is likely influenced by scheduling, promotions, opponent strength, and other game-day conditions.

In the next step lets check the attendance by day of the week. The `boxplot` from `pandas` library is used to make it easy for by level grouping for `day_of_week` variable. To void overlapping xticks were rotated by 45 degrees, also to overcome default title added by pandas boxplot, a suptitle was added and blank values were provided. the gridlines were also set to false to make sure the plot markers are visible and not overlapped in few cases.

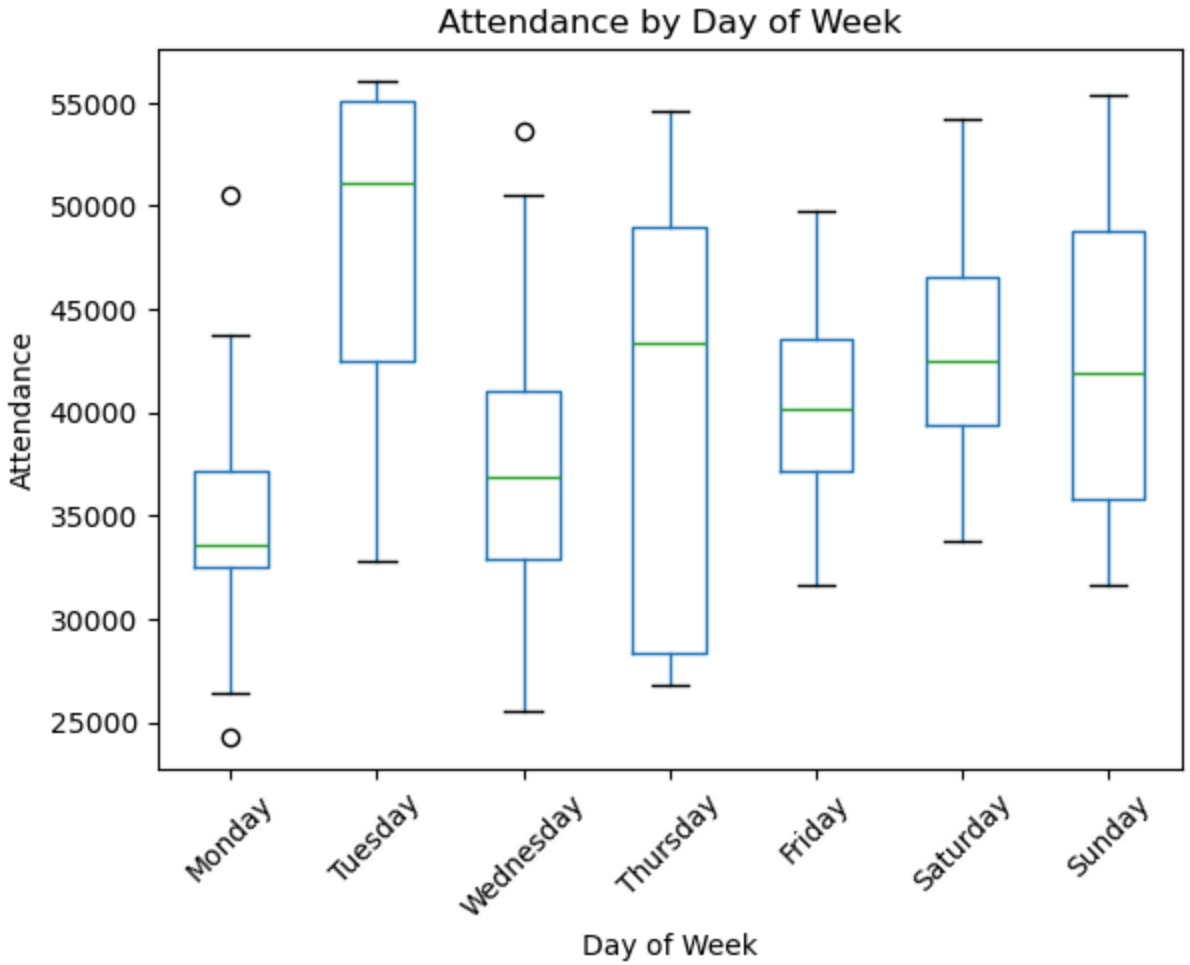
I had to add `day_order` list and then order the days in `mlb_data's` `day_of_week` variable to show them ordered in chart. There could be other better options available but as of now i felt this will work fine.

In [7]:
```
# order the days in dataset
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

mlb_data['day_of_week'] = pd.Categorical(
    mlb_data['day_of_week'],
    categories=day_order,
    ordered=True
)

# Attend by day_of_week distribution
plt.figure(figsize=(10, 6))
mlb_data.boxplot(column='attend', by='day_of_week', grid=False)
plt.title('Attendance by Day of Week')
plt.suptitle('')
plt.xlabel('Day of Week')
plt.ylabel('Attendance')
plt.xticks(rotation=45)
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



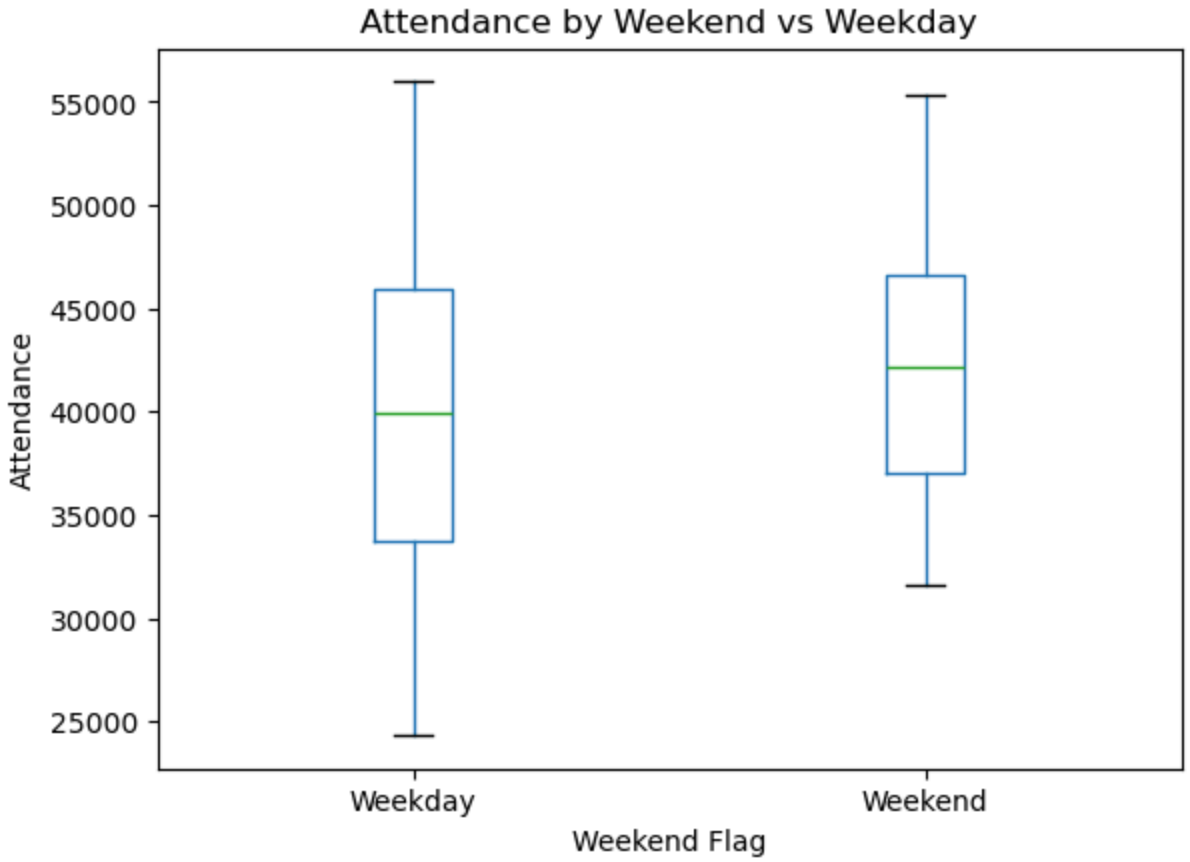
In above plot, the box plot indicates that attendance varies meaningfully by day of the week. Tuesday, Saturday, and Sunday games appear to draw stronger attendance overall, while Monday and Wednesday games tend to have lower median attendance. The variation within several days also suggests that day of week alone does not explain attendance, and that other factors such as promotions, opponents, and game conditions likely interact with scheduling.

In the next step lets create attendance distribution by weekday vs weekend. I will use the same code as used above but use the weekend_flag variable. Also, I have used xticks to label the plots correctly.

In [8]:
```
# attend by weekend vs weekday distribution
plt.figure(figsize=(8, 5))
```

```
mlb_data.boxplot(column='attend', by='weekend_flag', grid=False)
plt.title('Attendance by Weekend vs Weekday')
plt.suptitle('')
plt.xlabel('Weekend Flag')
plt.ylabel('Attendance')
plt.xticks([1, 2], ['Weekday', 'Weekend'])
plt.show()
```

```
<Figure size 800x500 with 0 Axes>
```



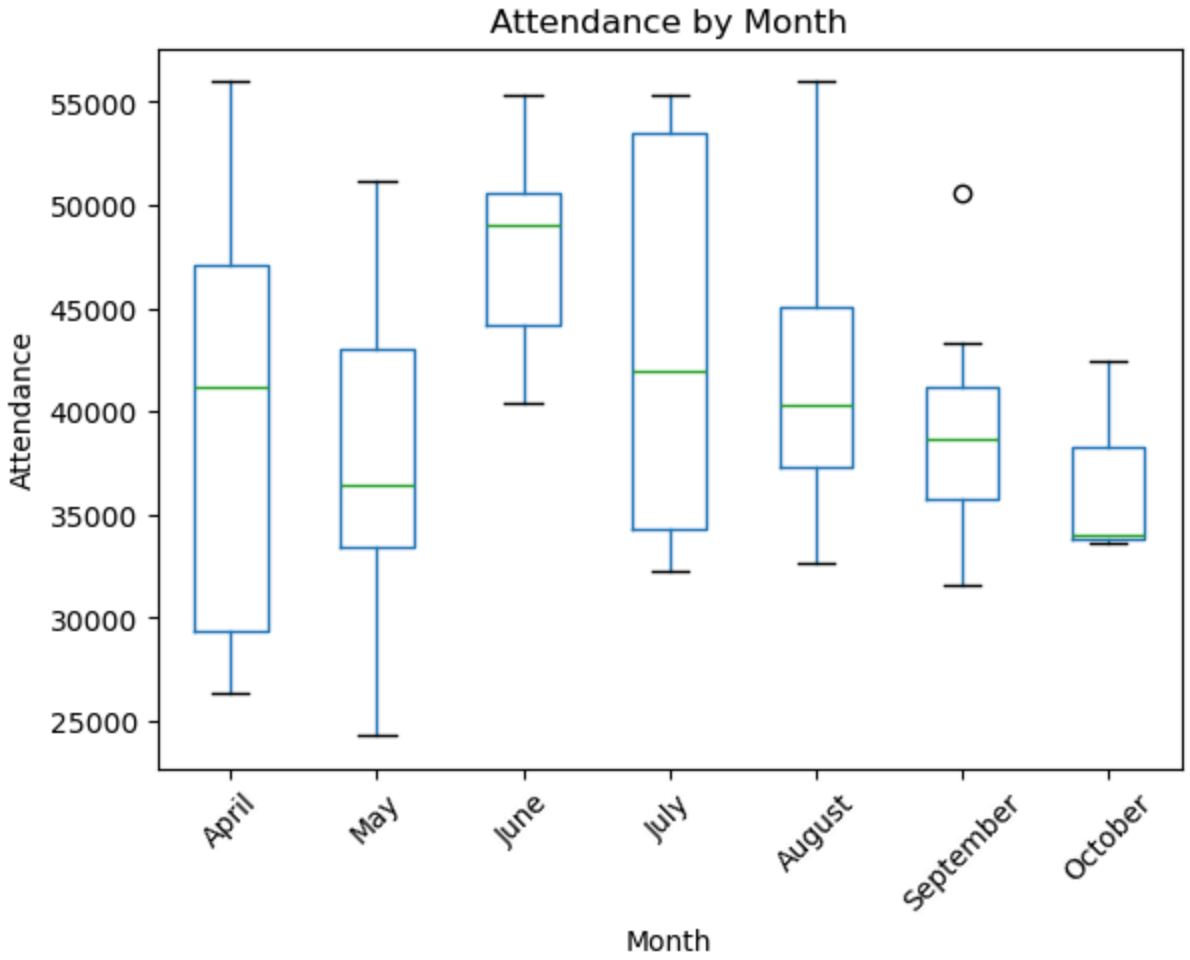
The weekend versus weekday box plot suggests that weekend games generally draw higher attendance than weekday games. Weekend games have a higher median attendance and a higher overall distribution, which supports the idea that fan availability on weekends improves turnout. However, the overlap between the two groups indicates that scheduling alone does not explain all attendance differences. Also, the lower end of weekday attendance appears weaker than the lower end of weekend attendance, which suggests that low-attendance games are more common during the week.

Next using the similar code, as above, lets check the distribution of attendance across different months. As we ordered days in week_of_days plot, we will do the same to order the months in calendar order.

In [9]:
```
# Order the months in dataset
month_order = [ 'April', 'May', 'June', 'July', 'August', 'September', 'October']
mlb_data['month'] = pd.Categorical(
    mlb_data['game_date'].dt.month_name(),
    categories=month_order,
    ordered=True
)

# Attendance by month distribution
plt.figure(figsize=(10, 6))
mlb_data.boxplot(column='attend', by='month', grid=False)
plt.title('Attendance by Month')
plt.suptitle('')
plt.xlabel('Month')
plt.ylabel('Attendance')
plt.xticks(rotation=45)
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



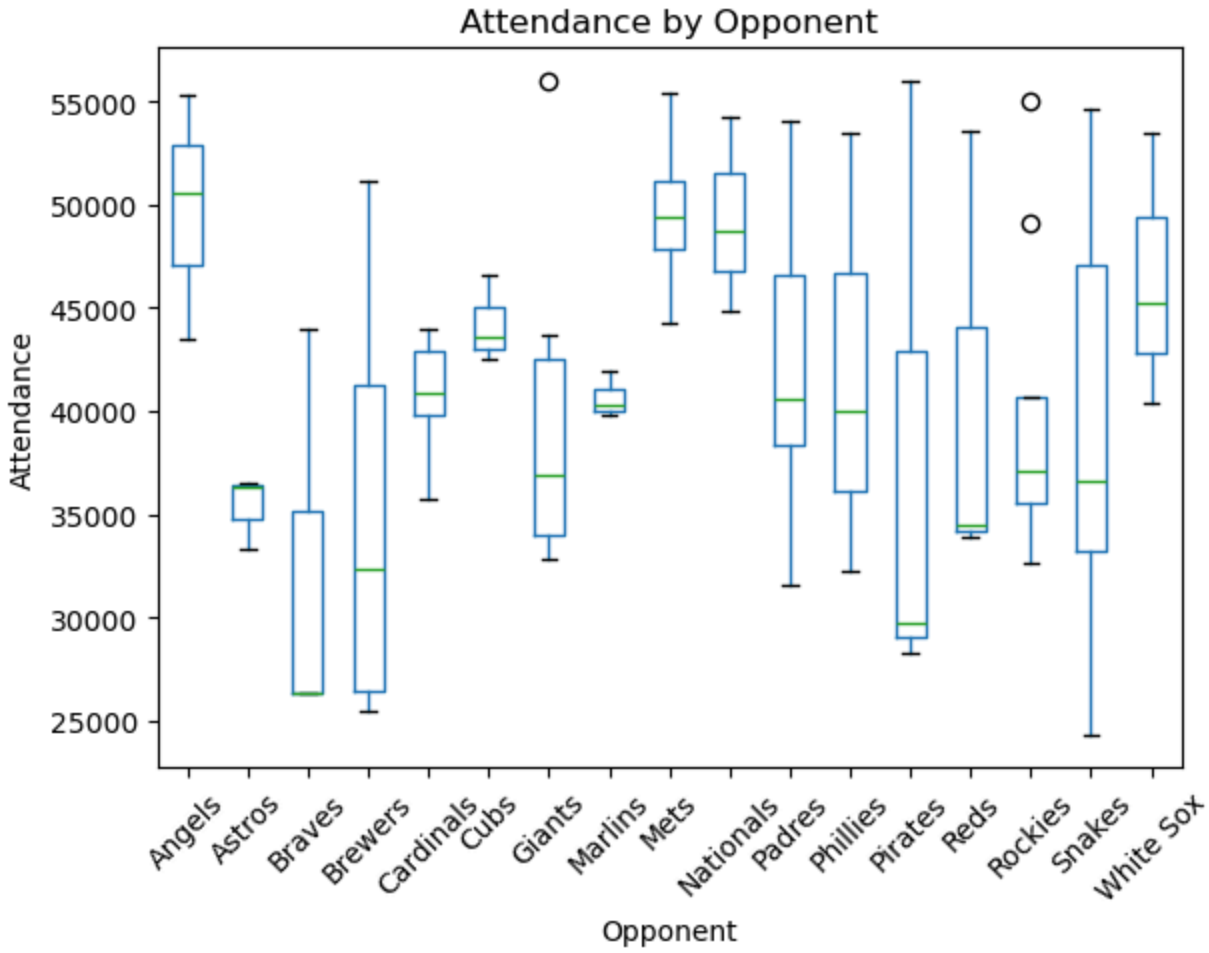
The above monthwise box plot suggests that attendance changes over the season. June shows the strongest typical attendance, while July also performs well but with greater variation from game to game. May and October appear comparatively weaker, and September includes at least one high-attendance outlier. Overall, the chart suggests that attendance tends to strengthen in the summer months, though seasonal timing alone does not fully explain the variation.

Now lets check the distribution by opponent. We will use the same code.

In [10]:
```
# attendance by opponent
plt.figure(figsize=(12, 6))
mlb_data.boxplot(column='attend', by='opponent', grid=False)
```

```
plt.title('Attendance by Opponent')
plt.suptitle('')
plt.xlabel('Opponent')
plt.ylabel('Attendance')
plt.xticks(rotation=45)
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```



The boxplot above indicates that some opponents are associated with consistently stronger attendance than others. Games against the `Angels` and `Mets` appear to draw the highest typical attendance, with relatively high median values. The `Cubs` and `White Sox` also seem to attract `solid crowds`. In contrast, opponents such as the `Pirates`, `Braves`, and

`Astros` appear to be associated with `lower` typical attendance levels. Several teams, including the Brewers, Padres, Phillies, and Snakes, show a wider spread in attendance, which suggests that crowd size against those opponents depends more heavily on other factors such as promotions, day of week, or season timing. A few high-attendance outliers are also visible for some opponents, indicating that certain matchups may have benefited from special circumstances beyond the opponent alone. I do want to mention that the game frequencies for some were low compared to others like Angels and Marlins and few others played just three games against dodgers.

Lets check how promotions helped the attendance. So first I created a list of all 4 promotions that were mentioned in dataset as `promo_cols`. Next created a 2X2 grid with a size of 12X8. Next I flattened the 2D array created for the subplot using `flatten()` function as it helps loop over the variables in for loop and assign the correct subplot. Also, in the for loop zip function is used to create the subplot with variable so it will pair [0,0] location to `cap` and [1,1] location to fireworks. for each subplot, two box plots were created, one when the promo was present and other when it was absent.

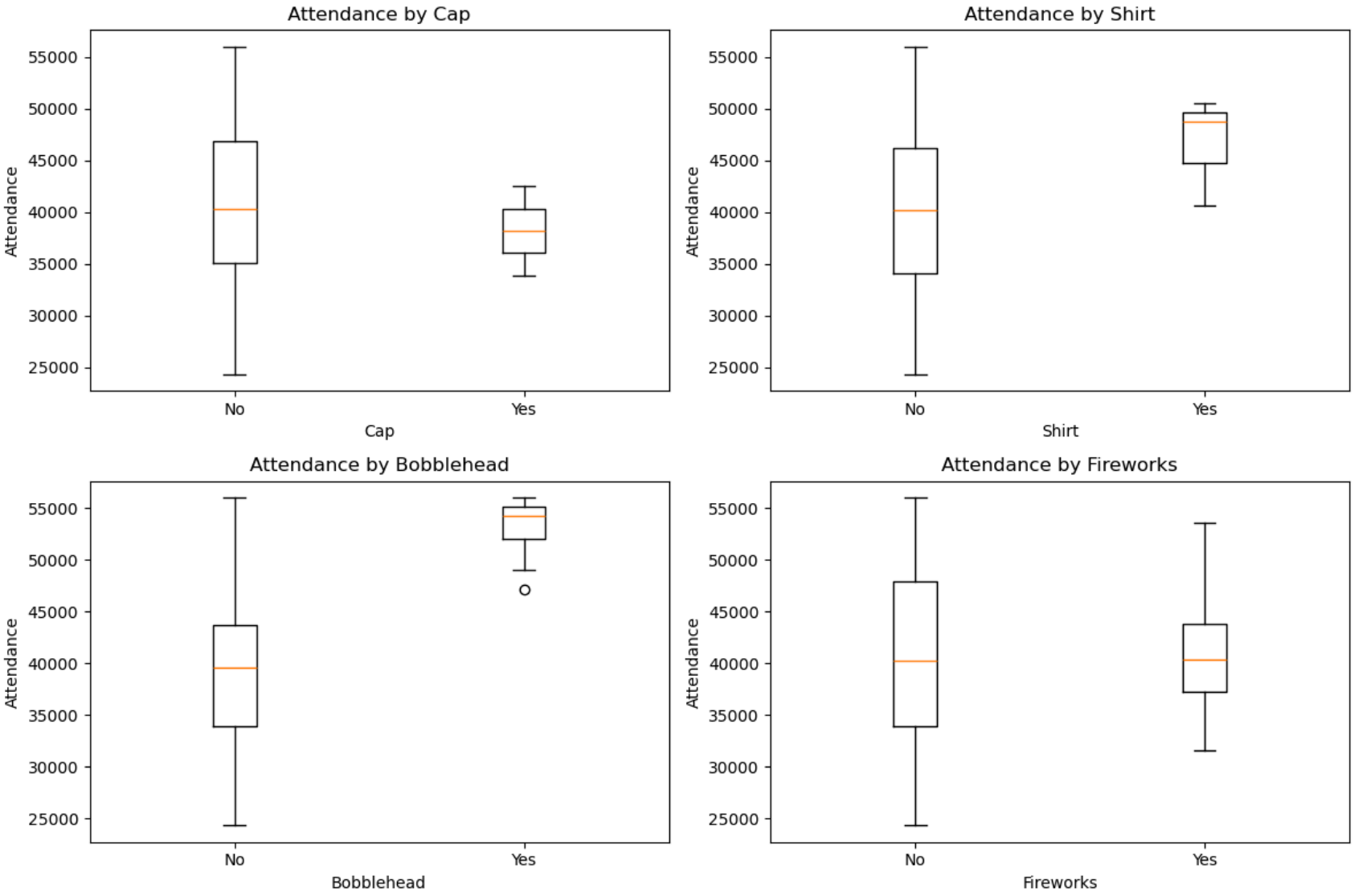
In [11]:
```
# attendance by cap, shirt, bobblehead, and fireworks
promo_cols = ['cap', 'shirt', 'bobblehead', 'fireworks']

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, promo in zip(axes, promo_cols):
    data = [
        mlb_data.loc[mlb_data[promo] == 'NO', 'attend'],
        mlb_data.loc[mlb_data[promo] == 'YES', 'attend']
    ]

    ax.boxplot(data, tick_labels=['No', 'Yes'])
    ax.set_title(f'Attendance by {promo.capitalize()}')
    ax.set_xlabel(promo.capitalize())
    ax.set_ylabel('Attendance')

plt.tight_layout()
plt.show()
```



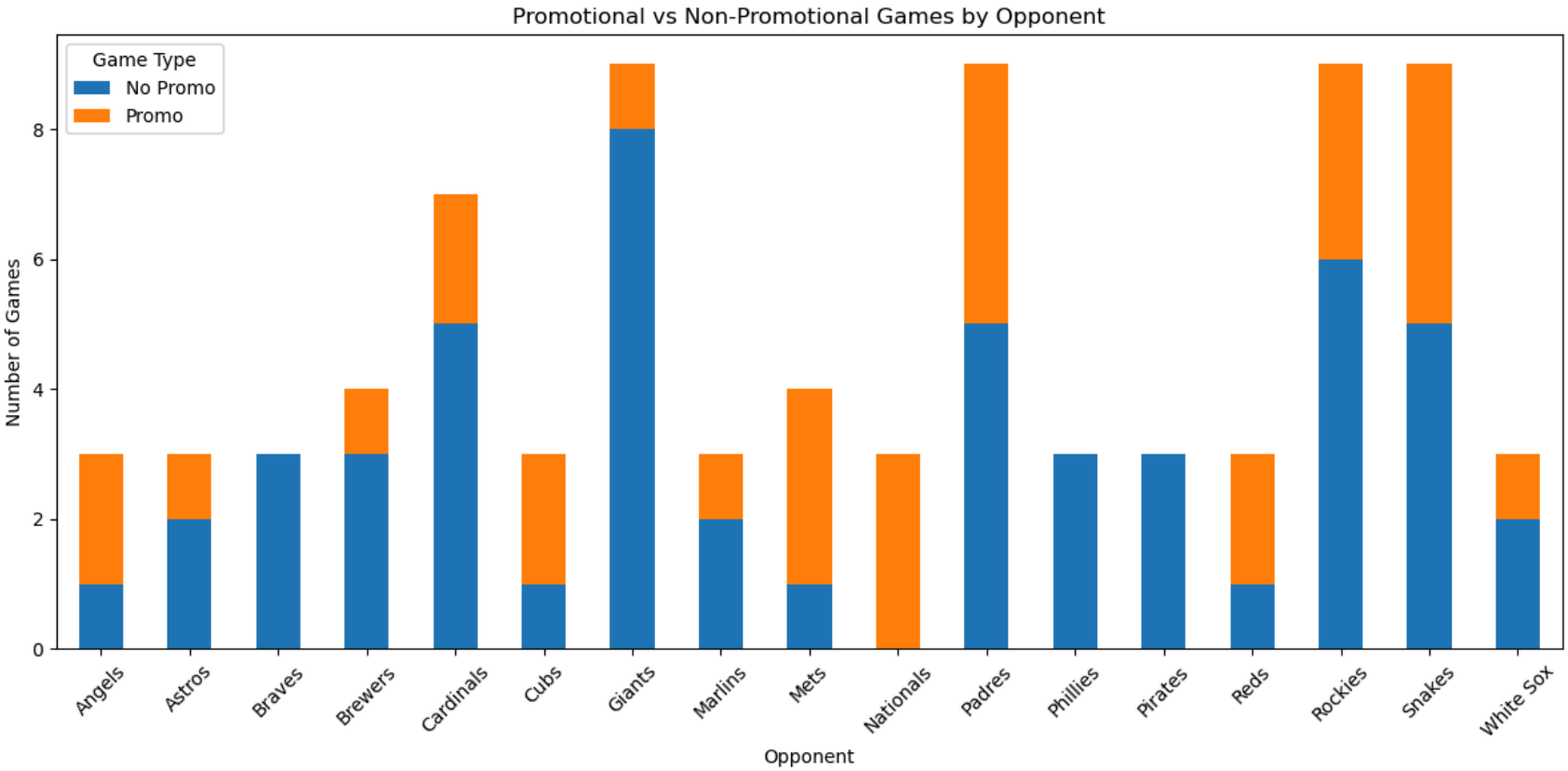
The box plots suggest that promotional games generally draw better attendance than non-promotional games. Bobblehead promotions appear to be associated with the strongest attendance lift, while shirt promotions also look effective but are based on fewer games. Fireworks show a smaller positive difference, and cap promotions appear to have the weakest separation. These results suggest that promotions may help attendance, although some of the observed differences may also overlap with factors such as weekends, opponents, and season timing.

Lets check how promotions overlap with the opponents. To do this, I first used `pd.crosstab()` to create a frequency table between the `opponent` column and the `promo_flag` column. This gives the number of games for each opponent split by whether a promotion was present or not. Since the `promo_flag` variable is stored as `True` and `False`, I renamed those column labels to `Promo` and `No Promo` so the chart would be easier to read. Then I plotted the table as a stacked bar chart, where each bar represents an opponent and each stacked section shows how many games against that opponent were promotional versus non-promotional. I also set the figure size to make the chart more readable, rotated the x-axis labels to avoid overlap, added a legend title, and used `tight_layout()` to improve spacing. This chart helps show whether promotions were evenly distributed across opponents or concentrated against specific teams.

In [12]:
```
# promotions with opponent
# promotions vs opponent: stacked bar chart
promo_opponent_counts = pd.crosstab(mlb_data['opponent'], mlb_data['promo_flag'])

promo_opponent_counts = promo_opponent_counts.rename(columns=
                                                     {False:'No Promo', True: 'Promo'})
promo_opponent_counts.plot(
    kind='bar',
    stacked=True,
    figsize=(12, 6)
)

plt.title('Promotional vs Non-Promotional Games by Opponent')
plt.xlabel('Opponent')
plt.ylabel('Number of Games')
plt.xticks(rotation=45)
plt.legend(title='Game Type')
plt.tight_layout()
plt.show()
```



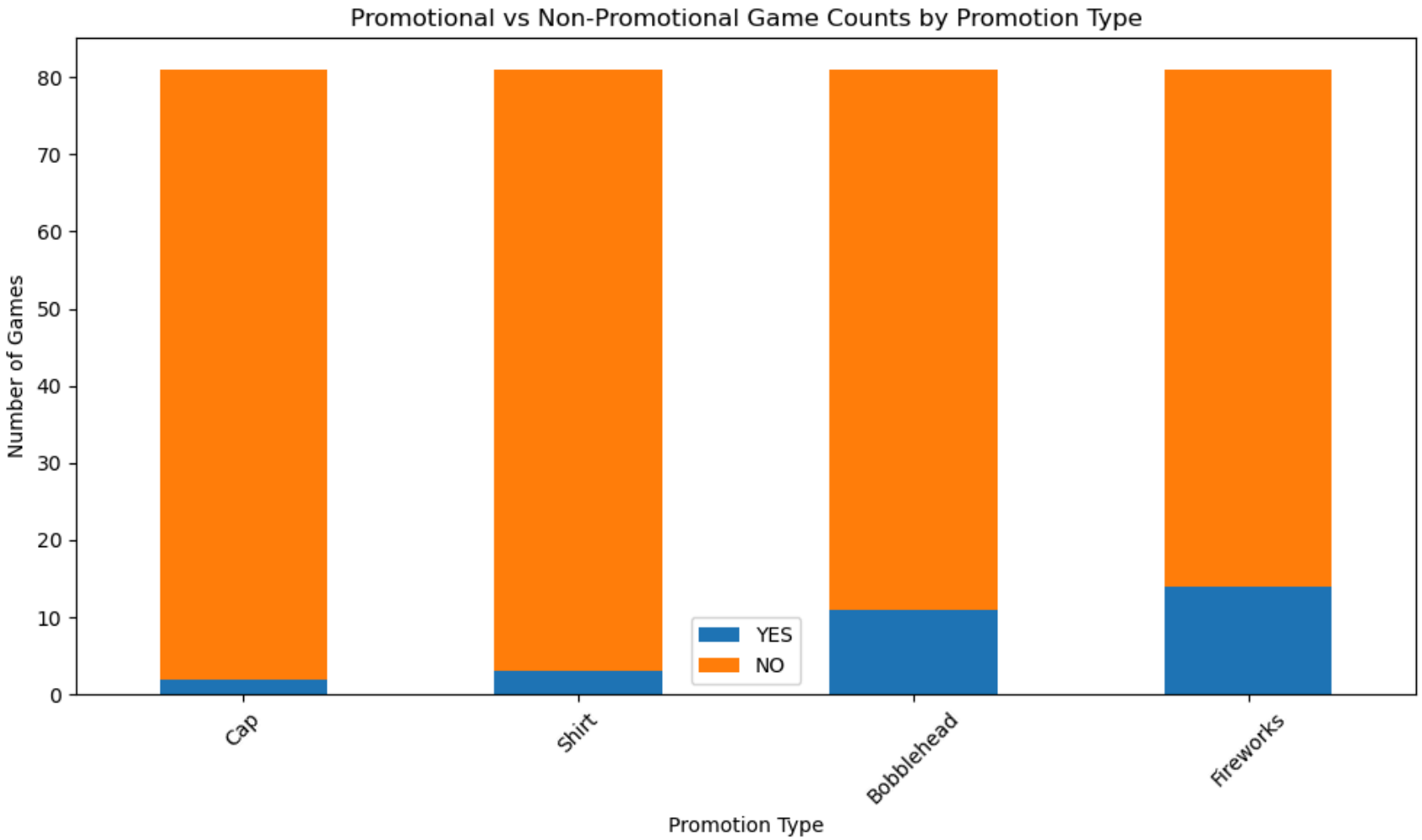
The chart shows that promotions were not distributed uniformly across all opponents. Some teams, such as the Snakes, Padres, Mets, and Nationals, appear to have a larger share of promotional games relative to others, while opponents such as the Braves, Phillies, and Pirates appear to have few or no promotional games in the dataset. The Giants and Rockies have many non-promotional games because they were played more often overall, but they still include some promotional events. This pattern is useful because it shows that promotions may overlap with certain opponents rather than being assigned randomly across all matchups. As a result, any attendance differences associated with promotions should be interpreted carefully, since part of the effect may also reflect the opponent being played.

Now to understand how frequently each promotion type was used, I created a new dataframe called promo_counts. In this data frame, I counted how many games each promotion marked as YES and how many times its marked as NO for the four promotion variables.

In [13]:
```
promo_counts = pd.DataFrame({
    'YES': [
        (mlb_data['cap'] == 'YES').sum(),
        (mlb_data['shirt'] == 'YES').sum(),
        (mlb_data['bobblehead'] == 'YES').sum(),
        (mlb_data['fireworks'] == 'YES').sum()
    ],
    'NO': [
        (mlb_data['cap'] == 'NO').sum(),
        (mlb_data['shirt'] == 'NO').sum(),
        (mlb_data['bobblehead'] == 'NO').sum(),
        (mlb_data['fireworks'] == 'NO').sum()
    ]
}, index=['Cap', 'Shirt', 'Bobblehead', 'Fireworks'])
```

The counts for YEs and NO were stored as two separate columns, and the promotion names were used as the row index so that each promotion type could be displayed clearly in the chart. After creating this summary table, I plotted it as a stacked bar chart. In the chart, each bar represents one promotion type, and the stacked sections show how many games included that promotion versus how many did not. I also set the figure size for readability, rotated the x-axis labels to avoid overlap, and used tight_layout() to improve spacing. This chart is useful because it shows whether some promotion types were used much more often than others, which helps explain why certain promotion effects may be more reliable than others.

In [14]:
```
promo_counts.plot(kind='bar', stacked=True, figsize=(10, 6))
plt.title('Promotional vs Non-Promotional Game Counts by Promotion Type')
plt.xlabel('Promotion Type')
plt.ylabel('Number of Games')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

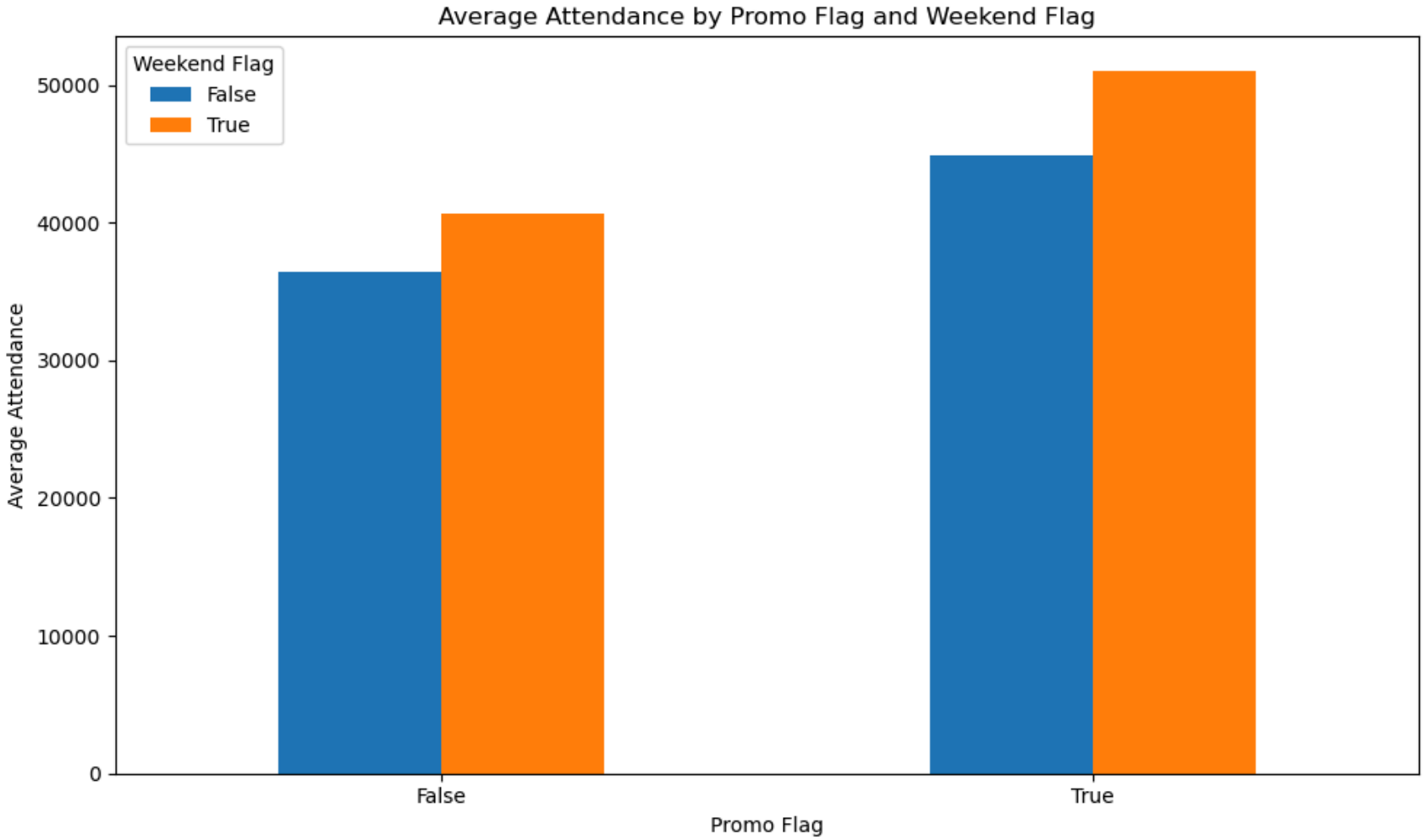


This stacked bar chart summarizes how often each promotion type was used during the season. Each bar represents one promotion category: Cap, Shirt, Bobblehead, and Fireworks. The blue portion shows the number of games where that promotion was offered YES, while the orange portion shows the number of games where it was not offered NO.The chart shows that all four promotion types were used in only a minority of games, since the NO portion is much larger than the YES portion for every category. Among the four, fireworks appears most frequently, followed by bobblehead promotions, while cap and shirt promotions were used the least often. This matters because promotion types with more YES games provide more evidence when comparing attendance patterns. In contrast, promotion types used only a few times should be interpreted more cautiously, since their apparent effect may be based on a small number of games rather than a consistent trend.

Lets add one more chart to better understand whether promotions are more effective on weekdays or weekends, I grouped the dataset using both promo_flag and weekend_flag. This creates four combinations of games: non-promotional weekdays, non-promotional weekends, promotional weekdays, and promotional weekends. After forming these groups, I selected the attend column and calculated the average attendance for each combination using mean(). I then used unstack() to reshape the grouped result into a table format so it could be plotted more clearly as a bar chart, with one category on the x-axis and the other shown through different colored bars.

After creating this summary table, I plotted it as a bar chart to compare the average attendance across those groups. The x-axis shows whether a game had a promotion or not, while the legend separates weekday and weekend games. I also set the figure size to make the chart easier to read, added a title and axis labels, kept the x-axis labels horizontal to avoid clutter, and used tight_layout() to improve spacing. This chart is useful because it does more than show whether promotions matter on their own. It also helps show whether promotions appear to be especially valuable on weekdays, where attendance is naturally weaker, compared with weekends, where attendance is already stronger.

In [15]:
```
# Average attendance by promo_flag split by weekend_flag
avg_attendance = mlb_data.groupby(['promo_flag', 'weekend_flag'])['attend'].mean().unstack()
avg_attendance.plot(kind='bar', figsize=(10, 6))
plt.title('Average Attendance by Promo Flag and Weekend Flag')
plt.xlabel('Promo Flag')
plt.ylabel('Average Attendance')
plt.xticks(rotation=0)
plt.legend(title='Weekend Flag')
plt.tight_layout()
plt.show()
```



This chart compares average attendance across promotional and non-promotional games, while also separating weekday and weekend games. The x-axis shows whether a game had any promotion promo_flag, and the colored bars show whether that game was played on a weekday or weekend weekend_flag. This makes the chart useful for understanding not only whether promotions are associated with higher attendance, but also whether that relationship changes depending on when the game is scheduled.The results suggest that both promotions and weekends are associated with stronger attendance. Games without promotions have lower average attendance overall, but even within that group, weekend games still attract more fans than weekday games. When promotions are present, attendance increases further, and the highest average attendance appears in promotional weekend games. At the same time, promotional weekday games also perform better than non-

promotional weekday games, which is especially important from a management perspective. This suggests that promotions may be most useful as a tool to strengthen weaker weekday games, while weekend games already benefit from naturally stronger demand.

Assumptions

I already explained most of the assumptions I made at the very start of the analysis but wanted to add a separate section here to include them altogether. First, although the dataset file is named as if it belongs to the 2022 season, the month, day, and weekday combinations align more closely with 2012, so the analysis assumes the correct year is 2012. Second, the attend variable is treated as the total attendance for each game and used as the main outcome of interest. Third, the temp variable is assumed to be recorded in Fahrenheit, and the holiday indicator includes only major U.S. federal holidays. Finally, this analysis is descriptive, so the patterns observed between attendance and factors such as promotions, opponents, weekends, and months should be interpreted as associations rather than proof of direct causation.

Conclusion The analysis suggests that Dodgers attendance is influenced most strongly by scheduling and season timing, with promotions acting as a useful secondary lever. Attendance was generally higher on weekends than weekdays, which supports the idea that fan availability plays a major role in turnout. A similar pattern appeared across the season, with summer months such as June and July showing stronger attendance than May or October. These findings indicate that natural demand is not evenly distributed and that some games are already advantaged by timing alone.

Opponent also appears to matter, as some teams were associated with stronger attendance than others. However, this effect should be interpreted carefully because several opponents were represented by only a small number of games. That makes opponent-level conclusions less stable than the broader patterns seen for weekends and months. Promotions showed a positive relationship with attendance as well, especially bobblehead and shirt promotions, while cap promotions appeared weaker. Fireworks were used more frequently and also showed some positive association, though not as strongly as bobbleheads.

The additional chart comparing average attendance by promotion and weekend status strengthens the management recommendation. Promotional weekday games performed better than non-promotional weekday games, which suggests that promotions may be most valuable when used on games that would otherwise draw weaker crowds. Weekend games already benefit from stronger baseline attendance, so adding major promotions to those games may produce less incremental value.

Overall, the best recommendation for management is to target stronger promotions toward lower-demand weekday games and weaker seasonal periods. In short, the Dodgers should use promotions strategically to lift weaker games rather than

concentrating them on games that are already likely to sell well.